

01 · Node.js 시작하기

백엔드 프레임워크 지형과 Node.js — 그리고 설치 없이 바로 쓰는 내장 모듈들

백엔드 프레임워크 지형



웹 백엔드 개발을 지원하는 다양한 언어와 프레임워크가 있습니다.

언어	대표 프레임워크	특징
Java	Spring Boot	대기업·금융권에서 널리 쓰임. 기능이 방대하고 구조가 엄격하다
JavaScript	Express, Nest.js	프론트와 같은 언어. 가볍고 생태계가 크다
Python	Django, FastAPI, Flask	Django는 관리자 페이지까지 포함한 풀스택, FastAPI는 비동기·타입 기반, Flask는 최소한만
Ruby	Ruby on Rails	관례를 따르면 작성할 코드가 확 줄어드는 설계
PHP	Laravel	호스팅 지원이 넓고 학습 자료가 많다
Go	Gin, Echo	컴파일 언어. 동시성 처리와 배포가 간단하다
C#	ASP.NET Core	마이크로소프트 생태계. 성능과 개발 도구가 강점

이름은 달라도 하는 일은 같습니다 — **HTTP 요청을 받고, 경로별로 나누고(라우팅), 공통 처리를 끼워 넣고(미들웨어), DB에 접근하고, 응답을 만든다.**

그래서 하나를 제대로 배우면 나머지는 용어만 갈아 끼우면 됩니다. 이 강의에서 배우는 개념들(라우팅·미들웨어·ORM·JWT)은 Spring에도 Django에도 그대로 있습니다.

Node.js란 무엇인가

JavaScript 런타임입니다. 프레임워크가 아니라, 브라우저 밖에서 JavaScript를 실행하는 환경입니다.

원래 JavaScript는 브라우저 안에서만 돌아왔습니다. 2009년 Ryan Dahl이 Chrome의 JavaScript 엔진인 **V8**을 꺼내 서버에서도 쓸 수 있게 만든 것이 Node.js입니다. 여기에 파일 시스템·네트워크처럼 브라우저에는 없던 기능을 붙였습니다.

Node.js ≠ Express. Node는 실행 환경이고, Express는 그 위에서 도는 프레임워크입니다. Java와 Spring의 관계와 같습니다.

핵심 구조 — Single Thread + Event Loop

전통적인 서버는 요청마다 스레드를 하나씩 만듭니다. 동시 접속이 1만 명이면 스레드도 1만 개고, 메모리와 컨텍스트 스위칭 비용이 커집니다.

Node는 스레드 하나가 이벤트 루프를 돌며 전부 처리합니다. 파일을 읽거나 DB 응답을 기다리는 동안 놀지 않고 다음 요청을 받습니다.

내용	
잘하는 것	I/O 중심 작업 — API 서버, 채팅, 실시간 알림, 스트리밍
못하는 것	CPU 집약 작업 — 이미지·영상 처리, 대규모 연산

다양한 JavaScript 런타임

런타임	공개	특징
Node.js	2009	가장 널리 쓰인다. 자료와 패키지가 압도적으로 많다
Deno	2018 (1.0은 2020)	Node를 만든 Ryan Dahl이 아쉬운 점을 고쳐 다시 만든 것. TypeScript 기본 지원, 보안 권한 모델
Bun	2022 (1.0은 2023)	속도에 집중. 번들러·테스트 러너·패키지 매니저가 하나로 통합

셋 다 서버에서 JavaScript를 돌립니다. 실무 채택률은 여전히 Node가 압도적이라 이 강의는 Node 기준입니다.

npm — 패키지 관리

npm(Node Package Manager)은 Node를 설치하면 함께 깔립니다. 두 가지를 뜻합니다.

- **공개 저장소** — 전 세계 개발자가 올린 패키지가 모인 곳(registry.npmjs.org)
- **CLI 도구** — 그 저장소에서 패키지를 받아 설치·관리하는 명령어

등록된 패키지 수는 다른 어떤 언어보다 많습니다. 필요한 기능은 대개 누군가 이미 만들어 두었습니다. 다만 그만큼 **품질 편차**가 크고 **의존성이 쉽게 불어납니다**.

package.json — 프로젝트의 명세서

`npm init -y`로 만듭니다. 이 파일 하나로 프로젝트가 무엇을 쓰는지 전부 드러냅니다.

```
{
  "name": "my-app",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "node --watch index.js"
  },
  "dependencies": {
    "express": "^4.21.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

필드	의미
<code>name</code> · <code>version</code>	패키지 이름과 버전. 공개 배포하지 않으면 크게 신경 쓰지 않아도 됩니다
<code>main</code>	이 패키지를 <code>require</code> 했을 때 로드될 진입 파일
<code>scripts</code>	<code>npm run <이름></code> 으로 실행할 명령 모음
<code>dependencies</code>	실행에 필요한 패키지 (express, pg ...)
<code>devDependencies</code>	개발에만 필요한 패키지 (테스트 도구, 린터 ...)
<code>type</code>	"module"이면 ESM(<code>import</code>), 없으면 CommonJS(<code>require</code>)
<code>private</code>	<code>true</code> 면 실수로 npm에 배포되는 것을 막습니다

`dependencies`와 `devDependencies` 구분은 배포 크기에 직결됩니다. 운영 서버에서는 `npm ci --omit=dev`로 개발용 패키지를 빼고 설치할 수 있습니다.

설치 명령

명령	하는 일
<code>npm install</code>	<code>package.json</code> 의 모든 의존성 설치
<code>npm install express</code>	설치하고 <code>dependencies</code> 에 기록
<code>npm install -D nodemon</code>	설치하고 <code>devDependencies</code> 에 기록
<code>npm install -g <pkg></code>	전역 설치. CLI 도구용 (nodemon, pm2 등)
<code>npm uninstall <pkg></code>	삭제하고 기록에서도 제거
<code>npm ci</code>	lock 파일 그대로 설치. <code>node_modules</code> 를 지우고 새로 만듭니다
<code>npx <pkg></code>	설치하지 않고 1회 실행 (<code>npx prisma studio</code>)

`npm install`과 `npm ci`의 차이가 실무에서 자주 문제가 됩니다.

- `npm install` — 버전 범위 안에서 최신 것을 받아올 수 있고, lock 파일을 갱신합니다
- `npm ci` — lock 파일에 적힌 **정확히 그 버전**만 설치합니다. `package.json`과 lock이 어긋나면 실패합니다

그래서 **CI/배포 파이프라인**에서는 반드시 `npm ci` 를 씁니다. "내 컴퓨터에서는 되는데"의 상당수가 이 차이에서 옵니다.

버전 표기 — semver(Semantic Versioning)

MAJOR.MINOR.PATCH 세 자리입니다. **4.21.1**이면 MAJOR 4, MINOR 21, PATCH 1.

자리	올라가는 때	
MAJOR	기존 코드가 깨지는 변경 (호환 불가)	
MINOR	기능 추가. 기존 코드는 그대로 동작	
PATCH	버그 수정	
표기	허용 범위	설명
<code>^4.21.1</code>	<code>>=4.21.1 <5.0.0</code>	MAJOR만 고정. npm 기본값
<code>~4.21.1</code>	<code>>=4.21.1 <4.22.0</code>	MINOR까지 고정. 더 보수적
<code>4.21.1</code>	정확히 그 버전	가장 엄격
<code>*/latest</code>	아무 버전	쓰지 마세요

`package-lock.json` — 반드시 커밋하세요

`package.json`은 "4.x면 아무거나"처럼 **범위**를 적습니다. 같은 `package.json`으로도 설치 시점에 따라 다른 버전이 깔릴 수 있다는 뜻입니다.

`package-lock.json`은 실제로 설치된 **정확한 버전**과 **의존성 트리 전체**를 기록합니다. 이 파일을 커밋해야 팀원과 서버가 완전히 같은 것을 설치합니다.

- `.gitignore`에 넣지 마세요. `node_modules`는 넣고, lock 파일은 **커밋**합니다
- 충돌이 나면 손으로 고치지 말고, 지운 뒤 `npm install`로 다시 만드세요
- `npm ci`는 이 파일만 봅니다

`scripts` — 명령 이름 붙이기

```
npm run dev           # scripts 의 dev 실행
npm start             # start 는 run 을 생략할 수 있다
npm test              # test 도 마찬가지
```

긴 명령을 이름으로 감싸 두면 팀원이 외울 것이 줄어듭니다. 이 저장소도 `npm run core:console`처럼 전 예제를 등록해 두었습니다.

`scripts` 안에서는 `node_modules/.bin`의 실행 파일을 경로 없이 부를 수 있습니다. 전역 설치가 필요 없는 이유입니다.

`node_modules`

설치된 패키지가 실제로 놓이는 폴더입니다. **절대 커밋하지 않습니다**(.gitignore에 넣습니다). package.json과 package-lock.json만 있으면 언제든지 똑같이 복원되기 때문입니다.

대안 — yarn, pnpm

도구	특징
npm	Node에 기본 포함. 따로 설치할 것이 없다
yarn	초기 npm이 느리던 시절 등장. workspaces(모노레포) 지원이 강점
pnpm	패키지를 한 곳에 두고 하드링크로 연결해 디스크와 설치 시간을 크게 절약

셋 다 같은 저장소를 씁니다. 이 강의는 추가 설치가 필요 없는 npm을 기준으로 합니다.

이 장에서 하는 일

여기까지가 배경이었습니다. 이제 Express 같은 프레임워크로 넘어가기 전에, **Node 자체가 기본으로 제공하는 것들**을 봅시다.

브라우저에 window와 document가 있듯이 Node에는 global과 process가 있고, 파일 시스템이나 운영체제 정보처럼 브라우저에서는 할 수 없던 일을 하는 **내장 모듈**이 들어 있습니다. require만 하면 바로 쓸 수 있고, 설치할 것이 없습니다.

여기서 배우는 path·fs·crypto는 뒤 장에서 계속 나옵니다. 파일 업로드(4장), 비밀번호 해싱(8장)이 전부 이 위에 얹힌 것입니다.

학습 목표

- package.json과 package-lock.json의 역할을 구분하고, npm install과 npm ci를 가려 쓴다
- semver 표기(^, ~)가 허용하는 버전 범위를 읽는다
- console의 로그 레벨과 table.time을 상황에 맞게 골라 쓴다
- setTimeout·setInterval·setImmediate의 차이를 이벤트 루프 단계로 설명한다
- process에서 실행 인자와 환경변수를 읽는다
- path로 운영체제에 안전한 경로를 만든다
- fs의 세 가지 스타일 중 서버에서 무엇을 써야 하는지 안다
- 단방향 해시와 양방향 암호화를 구분한다

사전 준비

Node.js 18 이상이면 됩니다. 설치할 패키지는 없습니다.

```
node -v    # v18.0.0 이상
```

1. console — 값 확인과 시간 측정

가장 많이 쓰는 디버깅 도구입니다. console.log만 쓰는 경우가 많지만, 목록 데이터는 table이, 성능 확인은 time/timeEnd가 훨씬 낫습니다.

`warn`과 `error`는 `stdout`이 아니라 `stderr`로 나갑니다. 로그를 파일로 리다이렉트할 때 `node app.js > log.txt` 하면 경고와 에러는 파일에 안 들어갑니다.

01-core/01-console.js

```
/**
 * console - 디버깅을 위한 값 확인, 표 출력, 실행 시간 측정
 *
 * console 은 global 객체의 프로퍼티라서 require 없이 바로 쓸 수 있다.
 * (브라우저의 window 에 해당하는 것이 Node 에서는 global / globalThis)
 *
 * 실행: node 01-core/01-console.js
 */

// ----- 1. 로그 레벨 -----
globalThis.console.log('Hello node.js!!'); // console.log 와 동일
console.info('정보 로그');
console.warn('경고 로그'); // stderr
console.error('에러 로그'); // stderr
console.debug('디버그 로그');

// ----- 2. 포맷 문자열 -----
const person = { name: '홍길동', age: 18, height: 180.5 };

// %s=문자열, %d=정수, %f=실수, %o=객체, %j=JSON
console.log(
  '이름은 %s, 나이는 %d, 키는 %f, 원본 %o',
  person.name,
  person.age,
  person.height,
  person
);

// ----- 3. 구조화 출력 -----
console.table([person, { ...person, name: '김철수', age: 20 }]);
console.dir(person, { depth: null, colors: true }); // 객체 속성을 재귀적으로

// ----- 4. 실행 시간 측정 -----
console.time(); // label 생략 시 'default'
let sum = 0;
for (let i = 1; i < 100000; i++) sum += i;
console.timeEnd();

console.time('작업1'); // label 을 주면 여러 구간을 동시에 측정 가능
for (let i = 1; i < 100000; i++) {}
console.timeEnd('작업1');

// ----- 5. 그룹 / 카운트 / 스택 -----
console.group('그룹 시작');
console.log('들여쓰기 된 로그');
console.groupEnd();

console.count('호출'); // 호출: 1
```

```
console.count('호출'); // 호출: 2

console.assert(sum > 0, '이 메시지는 조건이 false 일 때만 출력된다');
```

실행

```
node 01-core/01-console.js
```

실행 결과

```
Hello node.js!!
정보 로그
경고 로그
에러 로그
디버그 로그
이름은 홍길동, 나이는 18, 키는 180.5, 원본 { name: '홍길동', age: 18, height: 180.5
}



| (index) | name  | age | height |
|---------|-------|-----|--------|
| 0       | '홍길동' | 18  | 180.5  |
| 1       | '김철수' | 20  | 180.5  |



{ name: [32m'홍길동' [39m, age: [33m18 [39m, height: [33m180.5 [39m }
default: 2.483ms
작업1: 0.54ms
그룹 시작
  들여쓰기 된 로그
호출: 1
호출: 2
```

참고 할 것

- `console.table`은 배열/객체를 표로 그려 줍니다. 목록 확인에 `log`보다 낫습니다
- `console.dir(obj, { depth: null })`은 중첩 객체를 끝까지 펼칩니다
- `console.time(label)`에 라벨을 주면 여러 구간을 동시에 잴 수 있습니다
- 운영 코드에는 `console.log`를 남기지 말고 로거(winston, pino)를 쓰세요

2. timer — 그리고 setImmediate를 쓰는 이유

`setTimeout`과 `setInterval`은 브라우저와 같습니다. Node에만 있는 것이 `setImmediate`인데, 이게 왜 필요한지가 이 절의 핵심입니다.

Node는 싱글 스레드입니다. 무거운 작업을 한 번에 돌리면 그동안 서버가 요청을 못 받습니다. 작업을 잘게 쪼개 `setImmediate`로 이어 붙이면 각 조각 사이에 밀린 I/O가 처리되어 서버가 계속 응답할 수 있습니다.

01-core/02-timer.js

```

/**
 * timer - 일정 시간 뒤 / 주기적으로 콜백을 실행하는 전역 함수
 *
 * setTimeout(cb, ms)      : ms 이후 1회 실행          -> clearTimeout(id)
 * setInterval(cb, ms)     : ms 마다 반복 실행         -> clearInterval(id)
 * setImmediate(cb)        : 이벤트 루프 다음 턴에 즉시 실행 -> clearImmediate(id)
 *
 * 실행: node 01-core/02-timer.js
 */

setTimeout(() => {
  console.log('1초 후 실행');
}, 1000);

const timeout1 = setTimeout(() => {
  console.log('1.5초 후 실행');
}, 1500);

const timeout2 = setTimeout(() => {
  console.log('3초 후 실행 (2.5초에 취소되므로 절대 출력되지 않음)');
}, 3000);

const interval = setInterval(() => {
  console.log('1초마다 반복 실행');
}, 1000);

// 2.5초 뒤에 위 타이머들을 모두 정리한다.
setTimeout(() => {
  clearTimeout(timeout1); // 이미 실행된 타이머라 효과 없음
  clearTimeout(timeout2); // 아직 대기 중 -> 취소됨
  clearInterval(interval);
  console.log('타이머 정리 완료');
}, 2500);

setImmediate(() => {
  console.log('즉시 실행 (setImmediate)');
});

console.log('동기 코드가 항상 먼저 끝난다');

// ----- 위 주석의 마지막 주장을 직접 확인 -----
// I/O 콜백 안에서는 setImmediate 가 setTimeout(fn, 0) 보다 항상 먼저 실행된다.
// (poll 단계에서 I/O 콜백이 끝나면 바로 다음이 check 단계이기 때문)
require('fs').readFile(__filename, () => {
  setTimeout(() => console.log(' [I/O 안] setTimeout 0'), 0);
  setImmediate(() => console.log(' [I/O 안] setImmediate ← 항상 이쪽이 먼저'));
});

```

실행


```
node 01-core/02-timer.js
```

실행 결과

```
동기 코드가 항상 먼저 끝난다
즉시 실행 (setImmediate)
  [I/O 안] setImmediate ← 항상 이쪽이 먼저
  [I/O 안] setTimeout 0
1초 후 실행
1초마다 반복 실행
1.5초 후 실행
1초마다 반복 실행
타이머 정리 완료
```

3. process — 실행 환경 들여다보기

현재 실행 중인 Node 프로세스의 정보를 담은 전역 객체입니다. 실무에서 가장 자주 쓰는 것은 `process.env`(환경변수)와 `process.argv`(실행 인자)입니다.

01-core/03-process.js

```
/**
 * process - 현재 실행 중인 Node 프로세스와 시스템 정보를 담은 전역 객체
 *
 * 실행: node 01-core/03-process.js arg1 arg2
 */

console.log('version   : ', process.version); // Node 버전
console.log('arch      : ', process.arch); // CPU 아키텍처 (x64, arm64 ...)
console.log('platform  : ', process.platform); // 운영체제 (darwin, win32, linux)
console.log('pid       : ', process.pid); // 프로세스 ID
console.log('cwd()     : ', process.cwd()); // 실행한 위치(현재 작업 디렉터리)
console.log('uptime()  : ', process.uptime(), '초'); // 프로세스 구동 시간

console.log('memoryUsage(): ', process.memoryUsage());
// rss: 실제 할당된 물리 메모리, heapTotal/heapUsed: V8 힙, external: C++ 객체

// 환경변수 - dotenv 로 .env 파일 값을 여기에 주입한다.
console.log('NODE_ENV   : ', process.env.NODE_ENV || '(설정 안 됨)');
console.log('HOME       : ', process.env.HOME || process.env.USERPROFILE);

// 실행 인자: [0]=node 경로, [1]=스크립트 경로, [2]부터 사용자 인자
console.log('argv       : ', process.argv);
console.log('사용자 인자: ', process.argv.slice(2));

// 프로세스 종료 직전 후
process.on('exit', (code) => {
```

```
console.log(`프로세스 종료. exit code = ${code}`);
});

// process.exit(0); // 즉시 종료 (0=정상, 그 외=비정상)
```

실행

```
node 01-core/03-process.js arg1 arg2
```

실행 결과

```
version    : v22.22.3
arch       : arm64
platform   : linux
pid        : 39
cwd()      : /tmp/cap/01-core
uptime()   : 0.029277458 초
memoryUsage(): {
  rss: 43253760,
  heapTotal: 5365760,
  heapUsed: 3822184,
  external: 1421980,
  arrayBuffers: 10511
}
NODE_ENV   : (설정 안 됨)
HOME       : /sessions/magical-bold-fermi
argv       : [ '/usr/bin/node', '/tmp/cap/01-core/03-process.js' ]
사용자 인자: []
프로세스 종료. exit code = 0
```

참고 할 것

- `process.argv`는 `[0]`이 node 경로, `[1]`이 스크립트 경로입니다. 사용자 인자는 `slice(2)`
- `process.env`에 dotenv가 `.env` 값을 주입합니다
- `process.cwd()`는 명령을 실행한 위치, `__dirname`은 파일이 놓인 위치입니다
- 파일을 읽을 때는 `__dirname` 기준으로 해야 어느 폴더에서 실행하든 동작합니다

4. os — 시스템 정보

운영체제, CPU, 메모리 정보를 조회합니다. 실무에서는 `os.cpus().length`로 코어 수를 알아내 클러스터 모듈로 그만큼 프로세스를 띄우는 데 자주 씁니다.

01-core/04-os.js

```
/**
 * os - 운영체제, CPU, 메모리 정보를 조회하는 내장 모듈
 *
 */
```

```

* 실행: node 01-core/04-os.js
*/
const os = require('os');

console.log('<< 운영체제 정보 >>');
console.log('os.arch()      : ', os.arch()); // CPU 아키텍처
console.log('os.platform(): ', os.platform()); // darwin / win32 / linux
console.log('os.type()      : ', os.type()); // Darwin / Windows_NT / Linux
console.log('os.release()   : ', os.release()); // 커널 버전
console.log('os.hostname(): ', os.hostname()); // 컴퓨터 이름
console.log('os.uptime()   : ', os.uptime(), '초'); // 시스템 부팅 후 경과 시간
console.log('os.EOL       : ', JSON.stringify(os.EOL)); // 줄바꿈 문자 (\n 또는 \r\n)

console.log('\n<< 경로 정보 >>');
console.log('os.homedir(): ', os.homedir()); // 홈 디렉터리
console.log('os.tmpdir()  : ', os.tmpdir()); // 임시 디렉터리
console.log('process.cwd(): ', process.cwd()); // 현재 작업 디렉터리

console.log('\n<< CPU 정보 >>');
console.log('코어 개수:', os.cpus().length);
console.log('첫 번째 코어:', os.cpus()[0]);

console.log('\n<< 메모리 정보 >>');
const toGB = (b) => (b / 1024 ** 3).toFixed(2) + ' GB';
console.log('os.freemem() : ', toGB(os.freemem())); // 사용 가능 메모리
console.log('os.totalmem(): ', toGB(os.totalmem())); // 전체 메모리

```

실행

```
node 01-core/04-os.js
```

실행 결과

```

<< 운영체제 정보 >>
os.arch()      : arm64
os.platform(): linux
os.type()      : Linux
os.release()   : 6.8.0-124-generic
os.hostname(): claude
os.uptime()    : 159379.94 초
os.EOL         : "\n"

<< 경로 정보 >>
os.homedir(): /sessions/magical-bold-fermi
os.tmpdir()   : /sessions/magical-bold-fermi/tmp
process.cwd(): /tmp/cap/01-core

<< CPU 정보 >>

```

```
코어 개수: 4
첫 번째 코어: {
  model: 'unknown',
  speed: 0,
  times: { user: 114880, nice: 13850, sys: 222990, idle: 158859370, irq: 0
}
}
```

<< 메모리 정보 >>

```
os.freemem() : 3.32 GB
os.totalmem(): 3.81 GB
```

짚고 갈 것

- `os.platform()`은 `darwin`(macOS) / `win32` / `linux`를 돌려줍니다
- `os.EOL`은 줄바꿈 문자 — Windows는 `\r\n`, 그 외는 `\n`

5. path — 경로를 문자열로 잊지 마세요

Windows는 `\`, macOS/Linux는 `/`를 씁니다. 문자열 `+`로 경로를 만들면 한쪽에서만 동작하는 코드가 됩니다.

`join`과 `resolve`의 차이를 꼭 구분하세요. `join`은 조각을 이어붙여 정리하고, `resolve`는 오른쪽부터 `...`가 절대경로를 만나면 거기서 다시 시작합니다.

01-core/05-path.js

```
/**
 * path - 운영체제마다 다른 경로 구분자를 안전하게 다루는 내장 모듈
 *
 * 문자열을 직접 이어붙이지 말고 반드시 path.join / path.resolve 를 사용한다.
 * (Windows 는 \, macOS/Linux 는 / 를 쓰기 때문)
 *
 * 실행: node 01-core/05-path.js
 */
const path = require('path');

// __dirname : 현재 파일이 있는 폴더의 절대경로
// __filename : 현재 파일의 절대경로 (파일명 포함)
console.log('__dirname:', __dirname);
console.log('__filename:', __filename);

const filename = __filename;

console.log('\n<< 구분자 >>');
console.log('path.sep      :', path.sep); // 경로 구분자  / 또는 \
console.log('path.delimiter:', path.delimiter); // 환경변수 구분자 : 또는 ;

console.log('\n<< 경로 분해 >>');
console.log('dirname()  :', path.dirname(filename)); // 폴더 경로
console.log('basename() :', path.basename(filename)); // 파일명(확장자 포함)
console.log('extname()  :', path.extname(filename)); // 확장자
```

```

console.log(
  'basename - ext:',
  path.basename(filename, path.extname(filename)) // 확장자 뺀 파일명
);
console.log('parse()   :', path.parse(filename)); // 위 정보를 객체로
console.log('format()  :', path.format(path.parse(filename))); // parse 의
역

console.log('\n<< 경로 조합 >>');
// join: 상대경로를 그대로 이어붙이고 정리한다. .. 은 상위 폴더
console.log('join()    :', path.join(__dirname, '..', 'womanup', 'a.txt'));
// resolve: 오른쪽부터 훑다가 / 로 시작하는 절대경로를 만나면 거기서 멈춘다.
console.log('resolve():', path.resolve(__dirname, '..', '/web',
'womanup'));

console.log('\n<< 상대경로 / 절대경로 판별 >>');
console.log('isAbsolute():', path.isAbsolute(filename));
console.log('relative()  :', path.relative(__dirname, path.join(__dirname,
'..')));

console.log('\n<< OS 별 강제 지정 >>');
console.log('posix.sep  :', path.posix.sep);
console.log('posix.join:', path.posix.join('a', 'b', 'c.txt'));
console.log('win32.sep  :', path.win32.sep);
console.log('win32.join:', path.win32.join('a', 'b', 'c.txt'));

```

실행

```
node 01-core/05-path.js
```

실행 결과

```

__dirname : /tmp/cap/01-core
__filename: /tmp/cap/01-core/05-path.js

<< 구분자 >>
path.sep      : /
path.delimiter: :

<< 경로 분해 >>
dirname() : /tmp/cap/01-core
basename(): 05-path.js
extname() : .js
basename - ext: 05-path
parse()   : {
  root: '/',
  dir: '/tmp/cap/01-core',
  base: '05-path.js',
  ext: '.js',

```

```

    name: '05-path'
  }
  format() : /tmp/cap/01-core/05-path.js

<< 경로 조합 >>
join() : /tmp/cap/womanup/a.txt
resolve(): /web/womanup

<< 상대경로 / 절대경로 판별 >>
isAbsolute(): true
relative() : ..

<< OS 별 강제 지정 >>
posix.sep : /
posix.join: a/b/c.txt
win32.sep : \
win32.join: a\b\c.txt

```

6. fs — 세 가지 스타일과 서버에서의 선택

파일을 다루는 모듈입니다. 콜백·동기·프로미스 세 가지 스타일이 있는데, **서버 코드에서는 프로미스 방식을 씁니다.**

동기 방식(`readFileSync`)은 이벤트 루프를 멈춥니다. 서버 기동 시 설정 파일을 한 번 읽는 정도는 괜찮지만, 요청 처리 중에 쓰면 그동안 다른 모든 요청이 대기합니다.

01-core/06-fs.js

```

/**
 * fs - 파일 생성, 읽기, 수정, 삭제 등 파일 시스템 조작 모듈
 *
 * 세 가지 스타일이 있다.
 * 1) 콜백      : require('fs')           -> fs.readFile(path, cb)
 * 2) 동기      : require('fs')           -> fs.readFileSync(path)
 * 3) 프로미스  : require('fs').promises -> await fs.readFile(path) ← 실무 권장
 *
 * 실행: node 01-core/06-fs.js
 */
const fs = require('fs');
const fsp = require('fs').promises;
const path = require('path');

const readPath = path.join(__dirname, 'test.txt');
const writePath = path.join(__dirname, 'test_write.txt');

// ----- 1. 콜백 방식 -----
fs.readFile(readPath, (err, data) => {
  if (err) throw err;
  console.log('[콜백] Buffer :', data); // 인코딩을 안 주면 Buffer
  console.log('[콜백] toString:', data.toString()); // 문자열로 변환
});

```

```

fs.writeFile(writePath, '파일 쓰기 테스트\n', (err) => {
  if (err) throw err;
  console.log('[콜백] 파일 쓰기 완료');
});

// ----- 2. 동기 방식 -----
// 간단한 스크립트나 서버 기동 시 설정 로딩에만 쓴다.
// 요청 처리 중에 쓰면 이벤트 루프가 멈추므로 금지.
const syncData = fs.readFileSync(readPath, 'utf-8');
console.log('[동기] 내용:', syncData.trim());

// ----- 3. 프로미스 방식 (실무 권장) -----
(async () => {
  const data = await fsp.readFile(readPath, 'utf-8');
  console.log('[프로미스] 내용:', data.trim());

  await fsp.appendFile(writePath, '한 줄 더 추가\n'); // 이어쓰기
  console.log('[프로미스] 이어쓰기 완료');

  // 존재 확인 - fs.exists 는 deprecated, access 를 쓴다.
  try {
    await fsp.access(writePath, fs.constants.F_OK);
    console.log('[프로미스] 파일 존재함');
  } catch {
    console.log('[프로미스] 파일 없음');
  }

  // 디렉터리 목록 / 파일 정보
  console.log('[프로미스] 폴더 목록:', await fsp.readdir(__dirname));
  const stat = await fsp.stat(writePath);
  console.log('[프로미스] 크기:', stat.size, 'bytes / 디렉터리?',
stat.isDirectory());

  // 폴더 생성 (recursive: true 면 중간 경로까지, 이미 있어도 에러 없음)
  const tmpDir = path.join(__dirname, 'tmp');
  await fsp.mkdir(tmpDir, { recursive: true });

  // 정리
  await fsp.rm(writePath, { force: true });
  await fsp.rm(tmpDir, { recursive: true, force: true });
  console.log('[프로미스] 정리 완료');
})();

```

실행

```
node 01-core/06-fs.js
```

실행 결과

[동기] 내용: 안녕하세요. 이 파일은 fs 예제에서 읽어 들이는 샘플 텍스트입니다.

두 번째 줄입니다.

[콜백] 파일 쓰기 완료

[콜백] Buffer : <Buffer ec 95 88 eb 85 95 ed 95 98 ec 84 b8 ec 9a 94 2e 20 ec 9d b4 20 ed 8c 8c ec 9d bc ec 9d 80 20 66 73 20 ec 98 88 ec a0 9c ec 97 90 ec 84 9c 20 ec 9d bd ... 66 more bytes>

[콜백] toString: 안녕하세요. 이 파일은 fs 예제에서 읽어 들이는 샘플 텍스트입니다.

두 번째 줄입니다.

[프로미스] 내용: 안녕하세요. 이 파일은 fs 예제에서 읽어 들이는 샘플 텍스트입니다.

두 번째 줄입니다.

[프로미스] 이어쓰기 완료

[프로미스] 파일 존재함

[프로미스] 폴더 목록: [
 '01-console.js', '02-timer.js',
 '03-process.js', '04-os.js',
 '05-path.js', '06-fs.js',
 '07-url.js', '08-crypto.js',
 '09-module.js', '10-global.js',
 'test.txt', 'test_write.txt'

]

[프로미스] 크기: 43 bytes / 디렉터리? false

[프로미스] 정리 완료

7. url — 주소 분해하기

주소 문자열을 프로토콜·호스트·경로·쿼리로 나눕니다. 두 방식이 있는데 새 코드에서는 **new URL()** (WHATWG 표준) 을 쓰고, **url.parse()**는 기존 코드를 읽을 때만 알면 됩니다.

01-core/07-url.js

```
/**
 * url - 주소 문자열을 프로토콜/호스트/경로/쿼리로 분해하는 모듈
 *
 * 두 가지 방식이 있다.
 * 1) WHATWG URL (new URL) ← 현재 표준, 권장
 * 2) 레거시 url.parse()
 *
 * 실행: node 01-core/07-url.js
 */
const url = require('url');

const strURL =
  'http://wizard:password@apis.womanup.co.kr:8080/path1/path2' +
  '?serviceKey=key&base_date=20240407&dataType=JSON&numberOfRows=10' +
  '&pageNo=1&category=api&category=geo#urlTest';

// ----- 1. WHATWG URL (권장) -----
const myURL = new URL(strURL);
console.log('<< WHATWG URL >>');
```



```

console.log('protocol:', myURL.protocol); // http:
console.log('host      :', myURL.host); // 호스트 + 포트
console.log('hostname:', myURL.hostname); // 포트 제외
console.log('port      :', myURL.port);
console.log('pathname:', myURL.pathname);
console.log('search   :', myURL.search); // ?로 시작하는 쿼리 문자열
console.log('hash      :', myURL.hash); // #로 시작하는 프래그먼트

console.log('\n<< searchParams (URLSearchParams) >>');
const sp = myURL.searchParams;
console.log('get("category")   :', sp.get('category')); // 첫 번째 값만
console.log('getAll("category"):', sp.getAll('category')); // 같은 키 전부
console.log('has("pageNo")     :', sp.has('pageNo'));
console.log('keys()            :', [...sp.keys()]);
console.log('values()          :', [...sp.values()]);

console.log('\n<< 쿼리 수정 >>');
sp.append('fruit', 'apple'); // 추가 (중복 허용)
sp.append('fruit', 'banana');
console.log('append 후:', sp.getAll('fruit'));
sp.set('fruit', 'peach'); // 기존 값 전부 지우고 하나로 설정
console.log('set 후   :', sp.getAll('fruit'));
sp.delete('fruit'); // 삭제
console.log('delete 후:', sp.getAll('fruit'));
console.log('toString():', sp.toString());

// ----- 2. 레거시 url.parse (참고용) -----
console.log('\n<< 레거시 url.parse >>');
const urls = url.parse(strURL, true); // 두 번째 인자 true -> query 를 객체로
console.log('protocol:', urls.protocol);
console.log('host      :', urls.host);
console.log('query     :', urls.query); // { serviceKey: 'key', ... }
console.log('hash      :', urls.hash);

```

실행

```
node 01-core/07-url.js
```

실행 결과

```

<< WHATWG URL >>
protocol: http:
host      : apis.womanup.co.kr:8080
hostname: apis.womanup.co.kr
port      : 8080
pathname: /path1/path2
search   : ?
serviceKey=key&base_date=20240407&dataType=JSON&numberOfRows=10&pageNo=1&category=api&category=geo

```

```

hash      : #urlTest

<< searchParams (URLSearchParams) >>
get("category")    : api
getAll("category"): [ 'api', 'geo' ]
has("pageNo")      : true
keys()             : [
  'serviceKey',
  'base_date',
  'dataType',
  'numberOfRows',
  'pageNo',
  'category',
  'category'
]
values()           : [ 'key', '20240407', 'JSON', '10', '1', 'api', 'geo' ]

<< 쿼리 수정 >>
append 후: [ 'apple', 'banana' ]
set 후   : [ 'peach' ]
delete 후: []
toString():
serviceKey=key&base_date=20240407&dataType=JSON&numberOfRows=10&pageNo=1&category=api&category=geo

<< 레거시 url.parse >>
protocol: http:
host     : apis.womanup.co.kr:8080
query    : [Object: null prototype] {
  serviceKey: 'key',
  base_date: '20240407',
  dataType: 'JSON',
  numberOfRows: '10',
  pageNo: '1',
  category: [ 'api', 'geo' ]
}
hash     : #urlTest

```

8. crypto — 해시와 암호화

단방향 해시는 되돌릴 수 없습니다(비밀번호 저장, 무결성 검증). **양방향 암호화**는 키가 있으면 복호화됩니다(데이터 보관·전송).

비밀번호에 단순 해시만 쓰면 레인보우 테이블 공격에 뚫립니다. 그래서 사용자마다 다른 **salt**를 섞고, 해시를 수만 번 반복(key stretching)합니다. 실무에서는 이것 직접 구현하지 말고 **bcrypt(8장)**를 쓰세요.

01-core/08-crypto.js

```

/**
 * crypto - 해시, 난수, 대칭키 암호화를 제공하는 내장 모듈
 */

```

```

* 핵심 개념
* 단방향 해시: 되돌릴 수 없음. 비밀번호 저장, 무결성 검증에 사용
* 양방향 암호화: 키가 있으면 복호화 가능. 데이터 보관/전송에 사용
*
* 실행: node 01-core/08-crypto.js
*/
const crypto = require('crypto');

// ----- 1. 단방향 해시 -----
// 같은 입력 -> 항상 같은 출력. 조금만 달라도 완전히 다른 값이 나온다.
console.log('<< 단방향 해시 (sha512) >>');
console.log('base64:', crypto.createHash('sha512').update('비밀번호').digest('base64'));
console.log('hex   :', crypto.createHash('sha512').update('비밀번호').digest('hex'));
console.log('다른값:', crypto.createHash('sha512').update('비밀번호!').digest('base64'));

/*
알고리즘      출력 길이      현재 사용
MD5            128bit (16바이트)    X (충돌 발견)
SHA-1          160bit (20바이트)    X (충돌 발견)
SHA-256        256bit (32바이트)    0 (가장 널리 사용)
SHA-512        512bit (64바이트)    0
SHA-3          224~512bit          0 (SHA-2 후속)
*/

// ----- 2. salt + key stretching (pbkdf2) -----
// 해시만 쓰면 레인보우 테이블 공격에 뚫린다.
// salt          : 사용자마다 다른 임의 문자열을 섞어 같은 비밀번호도 다른 해시로
// key stretching: 해시를 수만 번 반복해 무차별 대입 비용을 올림
console.log('\n<< pbkdf2 (salt + 반복 해시) >>');
crypto.randomBytes(64, (err, buf) => {
  if (err) throw err;
  const salt = buf.toString('base64');
  console.log('salt:', salt.slice(0, 30) + '...');

  crypto.pbkdf2('password', salt, 100000, 64, 'sha512', (err2, key) => {
    if (err2) throw err2;
    console.log('hash:', key.toString('base64').slice(0, 30) + '...');
  });
});

// ----- 3. 양방향 대칭키 암호화 (AES) -----
console.log('\n<< 양방향 암호화 (aes-256-cbc) >>');
const algorithm = 'aes-256-cbc';
const key = 'abcdefghijklmnopqrstuvwxyz123456'; // 32바이트
const iv = '1234567890123456'; // 16바이트

const cipher = crypto.createCipheriv(algorithm, key, iv);
let encrypted = cipher.update('password', 'utf-8', 'base64');
encrypted += cipher.final('base64');
console.log('암호화:', encrypted);

```

```
const decipher = crypto.createDecipheriv(algorithm, key, iv);
let decrypted = decipher.update(encrypted, 'base64', 'utf-8');
decrypted += decipher.final('utf-8');
console.log('복호화:', decrypted);

// ----- 4. 안전한 랜덤 값 -----
console.log('\n<< 랜덤 값 >>');
console.log('randomUUID      :', crypto.randomUUID());
console.log('randomBytes(16)  :', crypto.randomBytes(16).toString('hex'));
```

실행

```
node 01-core/08-crypto.js
```

실행 결과

```
<< 단방향 해시 (sha512) >>
base64:
dvfV6nyLRRt3NxKSlTH0kkEGgqW2HRtfu190u/psUXvwlebbXCboxIPmDY0FRipqav2eUTBFuHa
Zri5x+usy1g==
hex    :
76f7d5ea7c8b451b773712929531ce92410682a5b61d1b5fbb5f4ebbfaf6c517bf095e6db5c2
6e8c483e60d8385448a6a6afd9e513045b87699ae2e71faeb32d6
다른값:
hp9b1MCfN1K659BaNPAG47wJXk2M5g0uC11Yq9duSkcA2zqhnfeGneFYkfWdJyHx0iX/DejeFun
tPAWjcd5eSw==

<< pbkdf2 (salt + 반복 해시) >>

<< 양방향 암호화 (aes-256-cbc) >>
암호화: DREAkvmqbQqoAce0B1PYw==
복호화: password

<< 랜덤 값 >>
randomUUID      : b588cd29-908b-4c31-915d-9748422f540a
randomBytes(16) : 2120c0312383015a3837031c680683e2
salt: ofZP7zEh1c9zV7YwL0+18BR09q8wj0...
hash: HVIUDv1LWNe5vSenWYKjDY6irdC2MT...
```

참고 할 것

- MD5·SHA-1은 충돌이 발견되어 쓰지 않습니다. SHA-256 이상을 쓰세요
- salt는 사용자마다 달라야 합니다. 같은 비밀번호도 다른 해시가 나오게 하는 장치입니다
- AES는 키 32바이트, IV 16바이트가 필요합니다
- `crypto.randomUUID()`로 고유 ID를, `randomBytes()`로 안전한 난수를 만듭니다

9. 모듈 시스템 — CommonJS vs ES Module

01-core/09-module.js

21 / 22

```
// math.mjs
export const add = (a, b) => a + b;
export default add;

// main.mjs
import add, { multiply } from './math.mjs';
import path from 'path';
import { fileURLToPath } from 'url';
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
*/
```

실행

```
node 01-core/09-module.js
```

실행 결과

```
직접 실행됨
add(2, 3)      = 5
multiply(2, 3) = 6
module.exports = { add: [Function: add], multiply: [Function: multiply] }
```